

番茄病虫害处置闭环管理系统 PRD

1. 项目名称

番茄病虫害处置闭环管理系统

可选名称：

- 番茄病虫害诊断与复查助手
- 家庭番茄异常处置系统
- 番茄病虫害诊断、处置与复查系统

本项目不以“Agent”作为产品名称，而是将 Agent 方法隐藏在系统内部，作为病例状态管理、动态决策、工具调用和复查闭环的实现机制。

2. 项目背景

家庭阳台种植、温室小规模种植、校园农业实践等场景中，用户经常会遇到番茄叶片发黄、斑点、虫害、果实异常等问题。

现有常见 AI 方案往往只解决“识别是什么病虫害”的问题，例如通过图片或文字判断“疑似早疫病”“疑似白粉虱”。但真实场景中，用户真正需要的是：

1. 现在信息够不够判断？
2. 这更像病害、虫害、缺素、肥害，还是环境问题？
3. 当前严重程度如何？
4. 应该马上处理，还是先观察？
5. 是否适合用药？
6. 距离采收较近时怎么办？
7. 做完处理后，什么时候复查？
8. 复查时如果好转/恶化，下一步应该怎么调整？
9. 整个处理过程能不能被记录下来？

因此，本项目不做单次问答型病虫害识别，而是做一个有状态的“病例处置闭环系统”。

3. 项目目标

本项目目标是构建一个面向番茄种植场景的智能处置系统，帮助用户完成从发现异常到复查结案的完整流程。

核心目标：

1. 支持用户创建番茄异常病例。
2. 系统根据症状信息判断是否需要补充信息。
3. 对常见番茄病虫害和非病虫害问题进行初步判断。
4. 生成以非化学措施优先的处置建议。
5. 根据采收期、用药历史、种植环境等进行安全约束。

6. 为每个病例创建复查任务。
 7. 用户复查后，系统判断症状趋势：好转、无变化、恶化。
 8. 根据复查结果调整病例状态，支持继续观察、升级建议或结案。
 9. 保存完整病例记录，便于后续查看。
-

4. 非目标范围

MVP 阶段明确不做以下内容：

1. 不做全作物覆盖，只做番茄。
 2. 不做专业农业生产级决策系统。
 3. 不提供具体农药名称、剂量、兑水比例、施药频次等强操作性用药处方。
 4. 不承诺医学/植保级诊断准确性。
 5. 不直接替代农技人员或专业植保人员。
 6. 不做复杂物联网环境监测接入。
 7. 不做真实农业监管合规系统。
 8. 不做完整图像识别模型训练；图片能力可作为后续增强项。
 9. 不做微信小程序、App、多端同步等复杂客户端功能。
 10. 不做自动购买农资、派单服务等商业闭环。
-

5. 目标用户

5.1 MVP 目标用户

主要面向以下用户：

1. 家庭阳台种植番茄的用户。
2. 校园农业实践、课程项目中的学生。
3. 小规模种植爱好者。
4. 对番茄病虫害有基础处理需求，但缺乏专业植保知识的人。

5.2 用户特点

这类用户通常：

1. 能描述叶片、果实、虫害等异常现象。
 2. 可能会上传照片，但不一定能提供高质量图像。
 3. 不清楚病害、虫害、缺素、肥害之间的区别。
 4. 需要明确的下一步处理建议。
 5. 更适合优先获得安全、保守、非化学处置建议。
 6. 需要复查提醒和复查判断，而不只是一次性答案。
-

6. 核心产品定位

本产品不是“番茄病虫害识别器”，而是：

面向番茄异常问题的诊断、处置、复查、结案闭环管理系统。

产品核心价值不是“识别得多准”，而是：

1. 不在信息不足时乱下结论。
 2. 能将一次咨询转化为可追踪病例。
 3. 能根据病例状态决定下一步动作。
 4. 能根据复查结果调整策略。
 5. 能用规则约束大模型输出，避免不安全建议。
 6. 能记录全过程，形成完整处置链路。
-

7. 典型用户场景

场景一：信息不足，系统主动追问

用户输入：

我的番茄叶子发黄，还有斑点，怎么办？

系统判断当前信息不足，不直接诊断，而是追问：

1. 发黄主要出现在老叶还是新叶？
2. 斑点是什么颜色？
3. 斑点有没有同心轮纹？
4. 叶背有没有霉层或小虫？
5. 番茄当前是苗期、开花期、结果期还是采收期？
6. 最近是否施肥或喷药？

系统状态更新为：

待补充信息

场景二：初步诊断并生成处置建议

用户补充：

主要是下部老叶，有褐色斑点，一圈一圈的，最近下过雨，现在结果期。

系统判断：

疑似番茄早疫病，可能性较高。

系统输出：

1. 判断依据。
2. 易混淆问题。
3. 立即处理建议。
4. 观察重点。

5. 升级条件。
6. 3 天后复查计划。

病例状态更新为：

待复查

场景三：临近采收，限制用药建议

用户输入：

还有 4 天就采收了，能不能用药？

系统判断存在采收安全风险，不输出具体药剂和剂量。

系统输出：

1. 不建议仅凭通用建议自行用药。
 2. 临近采收应优先采用非化学措施。
 3. 如必须用药，应咨询当地农技人员，并核对当地登记标签和采前安全间隔期。
 4. 当前可执行措施包括剪除重病叶、改善通风、避免叶面潮湿、隔离病叶等。
-

场景四：复查后病情恶化

三天后用户输入：

病斑变多了，上部叶子也开始有斑。

系统读取历史病例，判断：

1. 病斑数量增加。
2. 病害从下部叶片扩展到上部叶片。
3. 当前趋势为恶化。

系统输出：

1. 本次复查结果：恶化。
 2. 当前不建议继续单纯观察。
 3. 建议升级为专业确认。
 4. 如需进一步防治，应根据当地登记标签和农技人员建议执行。
 5. 病例状态更新为：建议人工确认。
-

场景五：复查后好转并结案

三天后用户输入：

剪掉病叶后没有新的斑点了，看起来好多了。

系统判断：

1. 新病斑未增加。
2. 症状未继续扩展。
3. 当前趋势为好转。

系统输出：

1. 继续保持通风和控水。
2. 继续观察 3-5 天。
3. 若无新症状，可结案。
4. 用户可点击“结案”。

病例状态更新为：

好转 / 可结案

8. 核心功能列表

8.1 病例创建

用户可以创建新的番茄异常病例。

输入字段：

1. 作物：默认番茄。
2. 种植环境：阳台、温室、露地。
3. 生长阶段：苗期、开花期、结果期、采收期、不确定。
4. 症状描述：文本输入。
5. 发生部位：老叶、新叶、叶背、茎、花、果实、整株、不确定。
6. 严重程度：少量、中等、大面积、不确定。
7. 最近天气：晴天、连续阴雨、高湿、高温、低温、不确定。
8. 最近是否施肥：无、有、不确定。
9. 最近是否用药：无、有、不确定。
10. 距离预计采收：天数，可为空。
11. 图片上传：MVP 可选，仅保存，不作为核心识别依据。

输出：

1. 创建病例。
 2. 返回病例 ID。
 3. 根据输入信息进入下一状态：待补充信息、已诊断、待复查等。
-

8.2 症状信息结构化

系统将用户自然语言描述转换为结构化信息。

示例输入：

下部老叶有褐色斑点，一圈一圈的，最近下雨后变多。

结构化输出：

```
{
  "affected_parts": ["下部老叶"],
  "symptom_keywords": ["褐色斑点", "同心轮纹", "雨后加重"],
  "possible_categories": ["真菌性叶部病害"],
  "missing_fields": ["距离采收时间", "叶背是否有霉层"],
  "severity": "不确定"
}
```

8.3 信息完整性判断

系统判断当前信息是否足够进行初步诊断。

需要关注的关键信息：

1. 是否明确作物。
2. 是否明确生长阶段。
3. 是否描述了主要症状。
4. 是否明确发生部位。
5. 是否存在病害、虫害、缺素、肥害的关键区分信息。
6. 是否涉及采收期安全问题。
7. 是否涉及近期用药或施肥。

如果信息不足，系统生成 3-5 个关键追问。

8.4 初步诊断

系统根据结构化症状和知识库内容，给出疑似问题排序。

MVP 支持的番茄问题：

1. 番茄早疫病。
2. 番茄晚疫病。
3. 番茄叶霉病。
4. 番茄灰霉病。
5. 白粉虱。
6. 蚜虫。
7. 红蜘蛛。
8. 脐腐病。
9. 缺镁黄化。
10. 肥害/药害。

诊断输出内容：

1. 疑似问题名称。
2. 可能性：较高、可能、较低。
3. 判断依据。
4. 易混淆问题。
5. 需要继续确认的信息。
6. 置信说明。

注意：不得输出“100%确定”“一定是”等绝对化表达。

8.5 安全约束检查

系统在生成处置建议前，必须执行安全检查。

安全检查规则：

1. 如果距离采收小于等于 7 天，不提供具体药剂、剂量、兑水比例等建议。
2. 如果用户近期已经用药，不提供混用建议。
3. 如果病害不确定，不提供确定性处方。
4. 如果用户为家庭阳台种植，优先给出非化学措施。
5. 如果症状快速扩展、大面积发生、果实受害，建议专业人员确认。
6. 如果用户要求具体农药和剂量，系统应拒绝直接给出处方，并提醒核对当地登记标签和咨询专业人员。

安全检查输出：

```
{
  "chemical_detail_allowed": false,
  "risk_level": "medium",
  "warnings": [
    "距离采收较近，不提供具体药剂和剂量建议",
    "如需用药，应核对当地登记标签和采前安全间隔期"
  ]
}
```

8.6 处置方案生成

系统根据诊断结果、安全检查、知识库内容生成处置方案。

处置方案分为以下部分：

1. 当前判断。
2. 严重程度。
3. 立即处理措施。
4. 非化学处置建议。
5. 观察重点。
6. 升级条件。

7. 安全提醒。
8. 复查计划。

示例输出结构：

```
{
  "summary": "当前更像番茄早疫病早期。",
  "severity": "轻度到中度",
  "immediate_actions": [
    "剪除严重病叶，并带离种植区域",
    "避免叶面喷水",
    "改善通风",
    "减少叶片长时间潮湿"
  ],
  "observation_points": [
    "是否出现新病斑",
    "病斑是否扩大",
    "是否扩展到上部叶片",
    "果实是否出现病斑"
  ],
  "escalation_conditions": [
    "3 天内病斑明显增加",
    "果实出现病斑",
    "多株同时快速发病"
  ],
  "safety_warnings": [
    "如需用药，应核对当地登记标签和采前安全间隔期"
  ],
  "followup_after_days": 3
}
```

8.7 复查任务创建

系统在给出处置方案后，为病例创建复查任务。

复查任务包含：

1. 复查日期。
2. 复查清单。
3. 上次诊断结果。
4. 上次建议执行项。
5. 当前状态。

默认复查时间：

1. 轻度问题：3 天后复查。
2. 中度问题：2-3 天后复查。
3. 快速扩展问题：1-2 天后复查，或建议人工确认。
4. 虫害问题：2-3 天后复查。

5. 缺素/肥害类问题：3-5 天后复查。

8.8 复查提交

用户可以在病例详情页提交复查结果。

输入字段：

1. 复查描述。
 2. 是否执行了上次建议。
 3. 是否有新病斑。
 4. 病斑是否扩大。
 5. 是否扩展到新部位。
 6. 是否出现果实受害。
 7. 是否上传新图片，可选。
-

8.9 复查趋势判断

系统根据上次病例记录和本次复查信息判断趋势。

趋势类型：

1. 好转。
2. 无明显变化。
3. 恶化。
4. 信息不足。
5. 需要人工确认。

判断逻辑示例：

1. 若用户描述“没有新病斑”“病斑没有扩大”“植株恢复”，判定为好转。
 2. 若用户描述“差不多”“没有明显变化”，判定为无明显变化。
 3. 若用户描述“更多了”“扩散到上部叶片”“果实也有斑”，判定为恶化。
 4. 若复查描述过短或不明确，继续追问。
 5. 若出现大面积枯萎、快速蔓延、果实严重受害，建议人工确认。
-

8.10 病例结案

当用户复查结果显示好转，且无新症状时，可以结案。

结案时系统保存：

1. 初始症状。
2. 初步诊断。
3. 处置方案。
4. 复查记录。
5. 最终结果。

6. 经验总结。

结案输出示例：

本病例已结案。初步判断为番茄早疫病早期，经剪除病叶、改善通风和减少叶面潮湿后，复查未见新病斑，当前建议继续保持通风管理。

9. 病例状态设计

9.1 状态枚举

MVP 使用以下状态：

NEED_MORE_INFO	待补充信息
DIAGNOSED	已初步诊断
FOLLOWUP_PENDING	待复查
FOLLOWUP_REVIEW	复查待判断
IMPROVING	好转
WORSENING	恶化
ESCALATED	建议人工确认
CLOSED	已结案

9.2 状态流转

```
新建病例
↓
NEED_MORE_INFO / DIAGNOSED
↓
FOLLOWUP_PENDING
↓
FOLLOWUP_REVIEW
↓
IMPROVING / WORSENING / ESCALATED / CLOSED
```

9.3 状态说明

NEED_MORE_INFO

当前信息不足，不能生成可靠诊断，需要用户补充。

DIAGNOSED

已形成初步诊断，但尚未生成或确认复查任务。

FOLLOWUP_PENDING

已生成处置建议，并创建复查任务，等待用户复查。

FOLLOWUP_REVIEW

用户已提交复查内容，系统需要判断趋势。

IMPROVING

症状好转，可继续观察或结案。

WORSENING

症状恶化，需要升级建议。

ESCALATED

建议用户联系当地农技人员或专业人员确认。

CLOSED

病例处理完成。

10. Agent 方法在系统中的体现

本项目内部采用 Agent 思路，但产品层面不刻意强调。

Agent 在系统中的角色不是“回答问题”，而是：

基于病例状态，决定下一步动作，并推进病例处置流程。

10.1 Agent 输入

每次用户提交信息时，Agent 接收：

1. 用户最新输入。
2. 当前病例状态。
3. 病例历史信息。
4. 已有诊断结果。
5. 安全约束结果。
6. 复查任务状态。
7. 可调用工具列表。

10.2 Agent 可选动作

ASK_MORE_INFO	追问信息
EXTRACT_SYMPTOMS	症状结构化
SEARCH_KNOWLEDGE	检索知识库
DIAGNOSE	初步诊断
CHECK_SAFETY	安全检查
GENERATE_PLAN	生成处置方案
CREATE_FOLLOWUP	创建复查任务

COMPARE_FOLLOWUP	判断复查趋势
ESCALATE	升级为人工确认
CLOSE_CASE	结案
SAVE_EVENT	保存事件记录

10.3 Agent 输出

Agent 每轮输出结构化决策：

```
{
  "next_action": "ASK_MORE_INFO",
  "reason": "缺少发生部位和生长阶段，无法区分病害、缺素或肥害",
  "tools_to_call": [],
  "state_update": "NEED_MORE_INFO",
  "user_response_type": "questions"
}
```

或：

```
{
  "next_action": "GENERATE_PLAN",
  "reason": "症状符合番茄早疫病早期特征，且安全检查通过",
  "tools_to_call": ["knowledge_search", "safety_check"],
  "state_update": "FOLLOWUP_PENDING",
  "user_response_type": "diagnosis_and_plan"
}
```

10.4 与普通大模型问答的区别

普通问答：

用户输入 → 大模型回答 → 结束

本系统：

用户输入
 → 读取病例状态
 → 判断下一动作
 → 必要时调用知识库/安全检查/历史记录
 → 生成受控建议
 → 更新病例状态
 → 创建复查任务
 → 后续根据复查结果继续调整

11. 页面需求

11.1 首页 / 病例列表页

功能：

1. 显示所有病例。
2. 按状态筛选病例。
3. 新建病例入口。
4. 继续复查入口。
5. 查看历史病例入口。

列表字段：

1. 病例标题。
2. 作物。
3. 疑似问题。
4. 当前状态。
5. 创建时间。
6. 复查时间。
7. 最近更新时间。

病例标题示例：

- 番茄下部叶片褐斑问题
- 番茄叶背小白虫问题
- 番茄果实底部发黑问题

11.2 新建病例页

表单字段：

1. 作物：默认番茄。
2. 种植环境：阳台、温室、露地。
3. 生长阶段：苗期、开花期、结果期、采收期、不确定。
4. 症状描述。
5. 发生部位。
6. 严重程度。
7. 最近天气。
8. 最近是否施肥。
9. 最近是否用药。
10. 距离采收时间。
11. 图片上传，可选。

提交后：

1. 若信息不足，进入补充问题页。
 2. 若信息足够，进入诊断结果页。
-

11.3 补充问题页

展示系统生成的追问。

功能：

1. 显示 3-5 个关键问题。
 2. 用户逐条回答或整体文本回答。
 3. 提交后继续诊断。
-

11.4 诊断结果页

展示内容：

1. 疑似问题排序。
 2. 判断依据。
 3. 易混淆问题。
 4. 置信说明。
 5. 处置建议入口。
-

11.5 处置方案页

展示内容：

1. 当前判断。
2. 严重程度。
3. 立即处理措施。
4. 非化学处置建议。
5. 观察重点。
6. 升级条件。
7. 安全提醒。
8. 复查时间。
9. 复查清单。

操作按钮：

1. 已了解并创建复查。
 2. 我想补充信息。
 3. 建议人工确认。
 4. 返回病例列表。
-

11.6 复查页

展示内容：

1. 上次诊断结果。
2. 上次处置建议。

3. 上次复查清单。
4. 复查输入框。
5. 是否执行建议。
6. 新图片上传，可选。

提交后：

1. 系统判断趋势。
2. 更新病例状态。
3. 给出下一步建议。

11.7 病例详情页

展示完整病例：

1. 基本信息。
2. 状态流转记录。
3. 用户每次输入。
4. 系统每次判断。
5. 处置方案。
6. 复查记录。
7. 最终结果。

12. 数据库设计

MVP 使用 SQLite。

12.1 cases 表

```
CREATE TABLE cases (  
  id INTEGER PRIMARY KEY AUTOINCREMENT,  
  title TEXT,  
  user_id TEXT,  
  crop TEXT,  
  environment TEXT,  
  growth_stage TEXT,  
  symptoms TEXT,  
  affected_parts TEXT,  
  severity TEXT,  
  recent_weather TEXT,  
  recent_fertilizer_use TEXT,  
  recent_pesticide_use TEXT,  
  days_to_harvest INTEGER,  
  suspected_problem TEXT,  
  likelihood TEXT,  
  status TEXT,  
  current_plan TEXT,  
  followup_date TEXT,
```

```
created_at TEXT,  
updated_at TEXT  
);
```

12.2 case_events 表

```
CREATE TABLE case_events (  
  id INTEGER PRIMARY KEY AUTOINCREMENT,  
  case_id INTEGER,  
  event_type TEXT,  
  user_input TEXT,  
  system_output TEXT,  
  structured_data TEXT,  
  created_at TEXT,  
  FOREIGN KEY(case_id) REFERENCES cases(id)  
);
```

event_type 可选值:

```
USER_SUBMIT  
ASK_MORE_INFO  
USER_REPLY  
DIAGNOSIS  
ACTION_PLAN  
FOLLOWUP_CREATED  
FOLLOWUP_SUBMITTED  
FOLLOWUP_RESULT  
ESCALATED  
CLOSED
```

12.3 followups 表

```
CREATE TABLE followups (  
  id INTEGER PRIMARY KEY AUTOINCREMENT,  
  case_id INTEGER,  
  due_date TEXT,  
  checklist TEXT,  
  status TEXT,  
  submitted_at TEXT,  
  user_description TEXT,  
  result TEXT,  
  created_at TEXT,  
  FOREIGN KEY(case_id) REFERENCES cases(id)  
);
```


status 可选值：

```
PENDING
SUBMITTED
DONE
CANCELLED
```

12.4 knowledge_entries 表，可选

MVP 可以直接使用 Markdown 文件，不一定入库。

如果入库，可设计：

```
CREATE TABLE knowledge_entries (
  id INTEGER PRIMARY KEY AUTOINCREMENT,
  crop TEXT,
  problem_name TEXT,
  category TEXT,
  symptoms TEXT,
  conditions TEXT,
  confusions TEXT,
  non_chemical_actions TEXT,
  escalation_conditions TEXT,
  safety_notes TEXT,
  created_at TEXT,
  updated_at TEXT
);
```

13. 知识库设计

MVP 使用 Markdown 文件管理知识库。

目录结构：

```
knowledge/
  tomato/
    early_blight.md
    late_blight.md
    leaf_mold.md
    gray_mold.md
    aphid.md
    whitefly.md
    spider_mite.md
    blossom_end_rot.md
```

```
magnesium_deficiency.md
fertilizer_burn.md
```

每个 Markdown 文件采用统一模板：

问题名称

问题类型

病害 / 虫害 / 缺素 / 肥害 / 环境问题

典型症状

- ...

常见发生部位

- ...

常见发生条件

- ...

易混淆问题

- ...

初步判断依据

- ...

非化学处置建议

- ...

观察重点

- ...

升级条件

- ...

安全提醒

- ...

14. API 需求

14.1 创建病例

POST /api/cases

请求:

```
{
  "crop": "番茄",
  "environment": "阳台",
  "growth_stage": "结果期",
  "symptoms": "下部老叶有褐色斑点, 有同心轮纹, 最近下雨后变多",
  "affected_parts": ["下部老叶"],
  "severity": "轻度",
  "recent_weather": "连续阴雨",
  "recent_fertilizer_use": "无",
  "recent_pesticide_use": "无",
  "days_to_harvest": 10
}
```

响应:

```
{
  "case_id": 1,
  "status": "FOLLOWUP_PENDING",
  "response_type": "diagnosis_and_plan",
  "diagnosis": {
    "suspected_problem": "番茄早疫病",
    "likelihood": "较高",
    "evidence": [
      "下部老叶出现褐色病斑",
      "病斑有同心轮纹",
      "雨后加重"
    ]
  },
  "plan": {
    "immediate_actions": [
      "剪除严重病叶并带离种植区域",
      "避免叶面喷水",
      "改善通风"
    ],
    "followup_after_days": 3
  }
}
```

14.2 补充信息

```
POST /api/cases/{case_id}/reply
```

请求:

```
{
  "message": "叶背没有霉层，主要是下部叶片，斑点是一圈一圈的。"
}
```

响应:

```
{
  "case_id": 1,
  "status": "FOLLOWUP_PENDING",
  "response_type": "diagnosis_and_plan",
  "message": "根据补充信息，当前更像番茄早疫病早期..."
}
```

14.3 获取病例详情

```
GET /api/cases/{case_id}
```

14.4 获取病例列表

```
GET /api/cases
```

可选查询参数:

```
status
keyword
created_from
created_to
```

14.5 提交复查

```
POST /api/cases/{case_id}/followup
```

请求：

```
{
  "description": "病斑变多了，上部叶片也开始有斑。",
  "actions_done": ["剪除病叶", "改善通风"],
  "has_new_spots": true,
  "spread_to_new_parts": true,
  "fruit_affected": false
}
```

响应：

```
{
  "case_id": 1,
  "trend": "worsening",
  "status": "ESCALATED",
  "recommendation": "与上次记录相比，症状正在扩展，建议联系当地农技人员确认。"
}
```

14.6 结案

```
POST /api/cases/{case_id}/close
```

请求：

```
{
  "summary": "复查未见新病斑，症状稳定。"
}
```

响应：

```
{
  "case_id": 1,
  "status": "CLOSED",
  "message": "病例已结案。"
}
```

15. 技术实现建议

15.1 推荐技术栈

MVP 推荐：

Python
FastAPI
SQLite
Pydantic
Jinja2 / Streamlit / Vue 任选其一
Markdown 知识库
LLM API

如果快速原型，建议：

Streamlit + SQLite + LLM API + Markdown 知识库

如果希望后续扩展为正式项目，建议：

FastAPI + Vue + SQLite/PostgreSQL + LLM API

15.2 后端目录结构建议

```
tomato-care-system/  
  backend/  
    app/  
      main.py  
    core/  
      config.py  
      database.py  
      llm.py  
    models/  
      schemas.py  
    services/  
      case_service.py  
      event_service.py  
      followup_service.py  
    agents/  
      decision_agent.py  
      diagnosis_agent.py  
      treatment_agent.py  
      followup_agent.py  
    tools/  
      knowledge_tool.py  
      safety_tool.py  
      weather_tool.py  
      case_tool.py  
    prompts/  
      symptom_extract.md  
      decision.md
```

```
diagnosis.md
treatment.md
followup_compare.md
knowledge/
tomato/
  early_blight.md
  late_blight.md
  leaf_mold.md
  gray_mold.md
  aphid.md
  whitefly.md
  spider_mite.md
  blossom_end_rot.md
  magnesium_deficiency.md
  fertilizer_burn.md
data/
  app.db
frontend/
README.md
```

16. Prompt 设计要求

16.1 症状结构化 Prompt

目标：将用户自然语言转换为结构化 JSON。

要求：

1. 不做诊断结论。
2. 只提取症状、部位、阶段、严重程度、缺失字段。
3. 输出必须是 JSON。
4. 不允许输出 Markdown。

输出示例：

```
{
  "crop": "番茄",
  "environment": "阳台",
  "growth_stage": "结果期",
  "affected_parts": ["下部老叶"],
  "symptoms": ["褐色斑点", "同心轮纹"],
  "severity": "轻度",
  "missing_fields": ["距离采收时间"],
  "possible_categories": ["真菌性叶部病害"]
}
```

16.2 决策 Prompt

目标：根据当前病例状态选择下一步动作。

输入：

1. 当前病例状态。
2. 结构化症状。
3. 历史事件。
4. 可用工具。
5. 安全规则摘要。

输出：

```
{
  "next_action": "ASK_MORE_INFO",
  "reason": "缺少发生部位和生长阶段，无法判断",
  "questions": [
    "斑点主要出现在老叶还是新叶？",
    "叶背是否有霉层或虫体？"
  ],
  "tools_to_call": [],
  "state_update": "NEED_MORE_INFO"
}
```

16.3 诊断 Prompt

目标：根据结构化症状和知识库内容生成初步诊断。

要求：

1. 必须给出疑似问题，而不是绝对结论。
2. 必须给出判断依据。
3. 必须列出易混淆问题。
4. 必须保留不确定性。
5. 不提供具体农药处方。

16.4 处置方案 Prompt

目标：生成受安全规则约束的处置建议。

要求：

1. 优先非化学措施。
2. 不输出具体农药剂量。
3. 如果 Safety Checker 禁止具体用药建议，必须遵守。
4. 必须包含复查计划。

5. 必须包含升级条件。

16.5 复查比较 Prompt

目标：比较历史病例和当前复查描述，判断趋势。

输出：

```
{
  "trend": "worsening",
  "evidence": [
    "用户描述病斑变多",
    "扩展到上部叶片"
  ],
  "next_action": "ESCALATE",
  "state_update": "ESCALATED"
}
```

17. 安全与免责声明

系统必须在合适位置展示以下原则：

1. 本系统仅提供家庭种植和学习场景下的辅助建议。
 2. 本系统不能替代当地农技人员、植保专家或农药标签说明。
 3. 涉及农药使用时，应以当地登记标签、产品说明和专业人员建议为准。
 4. 临近采收、病害快速扩展、大面积受害、果实受害等情况，应优先咨询专业人员。
 5. 系统不提供具体农药剂量、混配方案和高风险操作指导。
-

18. MVP 验收标准

MVP 完成后，应满足以下标准：

18.1 病例流程

1. 用户可以新建病例。
2. 系统可以保存病例。
3. 系统可以根据输入判断是否追问。
4. 系统可以生成初步诊断。
5. 系统可以生成处置方案。
6. 系统可以创建复查任务。
7. 用户可以提交复查。
8. 系统可以判断复查趋势。
9. 系统可以更新病例状态。
10. 用户可以查看历史病例。

18.2 Agent 决策

1. 系统不是每次都直接回答。
2. 信息不足时会追问。
3. 信息足够时会诊断。
4. 存在采收安全风险时会限制输出。
5. 复查恶化时会升级建议。
6. 症状好转时支持继续观察或结案。

18.3 安全输出

1. 不输出具体农药剂量。
2. 不输出确定性诊断。
3. 不建议用户盲目用药。
4. 临近采收时必须提示安全间隔期。
5. 病情严重时必须建议专业确认。

18.4 数据持久化

1. 病例数据持久化保存。
2. 事件记录持久化保存。
3. 复查任务持久化保存。
4. 结案病例可回看全过程。

19. 第一阶段开发计划

阶段一：基础病例系统

目标：

1. 创建病例。
2. 保存病例。
3. 查看病例列表。
4. 查看病例详情。
5. 更新病例状态。

暂不接入 LLM。

阶段二：规则版状态机

目标：

1. 手写规则判断信息是否足够。
 2. 手写规则判断是否进入复查。
 3. 手写规则判断复查好转/恶化。
 4. 跑通完整闭环。
-

阶段三：接入 LLM 症状结构化

目标：

1. 用户自然语言输入。
 2. LLM 输出结构化 JSON。
 3. 系统根据结构化字段继续流程。
-

阶段四：接入知识库诊断

目标：

1. 建立番茄知识库 Markdown。
 2. 根据症状关键词检索相关知识。
 3. LLM 生成疑似问题和依据。
-

阶段五：接入处置方案生成

目标：

1. 加入 Safety Checker。
 2. 生成受控处置建议。
 3. 自动创建复查任务。
-

阶段六：复查闭环

目标：

1. 用户提交复查。
 2. 系统对比历史记录。
 3. 判断好转、无变化、恶化。
 4. 更新状态。
 5. 支持结案。
-

20. 后续扩展方向

MVP 完成后，可扩展：

1. 接入天气 API。
2. 接入图片识别或多模态模型。
3. 支持更多作物，如黄瓜、辣椒、草莓。
4. 支持用户种植档案。
5. 支持复查提醒。
6. 支持知识库向量检索。
7. 支持病例总结报告导出。

8. 支持本地化病虫害知识。
9. 支持小程序端。
10. 支持温室环境数据接入。

21. 项目成功标准

本项目成功的标志不是“能回答番茄病虫害问题”，而是：

1. 系统能把一次用户咨询转成一个病例。
2. 系统能根据病例状态决定下一步。
3. 系统能追问、诊断、建议、复查、升级或结案。
4. 系统能保存完整处置过程。
5. 系统输出是保守、安全、有约束的。
6. 系统展示出“有状态任务推进”的能力，而不是普通聊天问答。

22. 给开发者的核心理解

开发时不要把系统做成：

用户问一句 → LLM 回一句

应该做成：

用户输入
→ 读取病例状态
→ 判断下一步动作
→ 调用必要模块
→ 生成受控输出
→ 更新病例状态
→ 保存事件记录
→ 推进复查闭环

这是本项目最核心的产品逻辑。